Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Middleware

L'histoire d'une application qui grandit

C'est quoi un middleware ?

- Un logiciel qui se place **entre** notre application et le reste du monde
- Ce n'est pas notre code, mais notre application ne marche pas sans
- Exemples : Nginx, PostgreSQL, RabbitMQ

FraisDirect

Une épicerie en ligne qui livre des produits frais



Un serveur, une application, une base. Tout va bien… pour l'instant.

Au programme

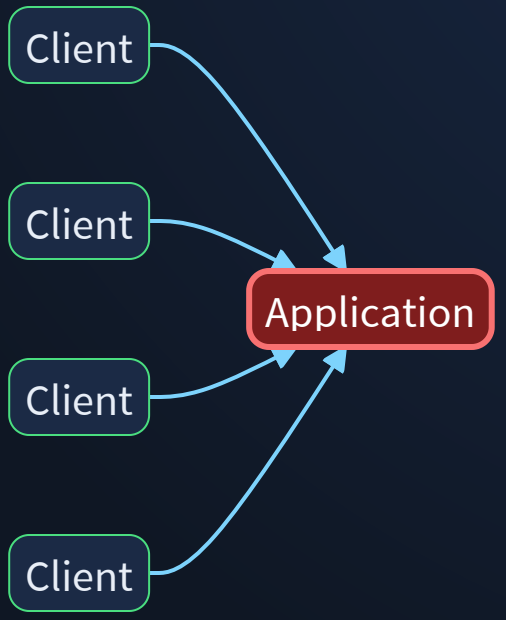
1. Le site est saturé : **load balancer**
2. Plusieurs applications, une seule adresse : **reverse proxy**
3. Des données de toutes les formes : **bases de données**
4. Des services qui doivent se parler : **messaging**

ACTE 1

Le Black Friday

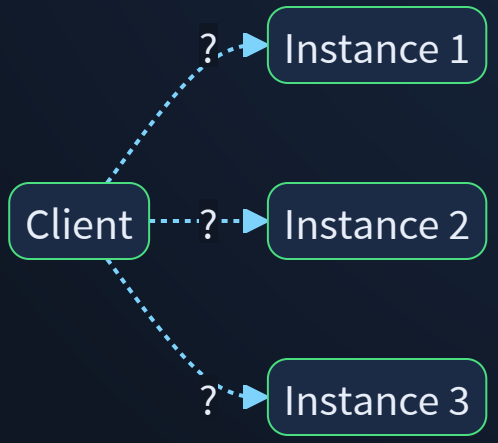
Le site est saturé

Le problème



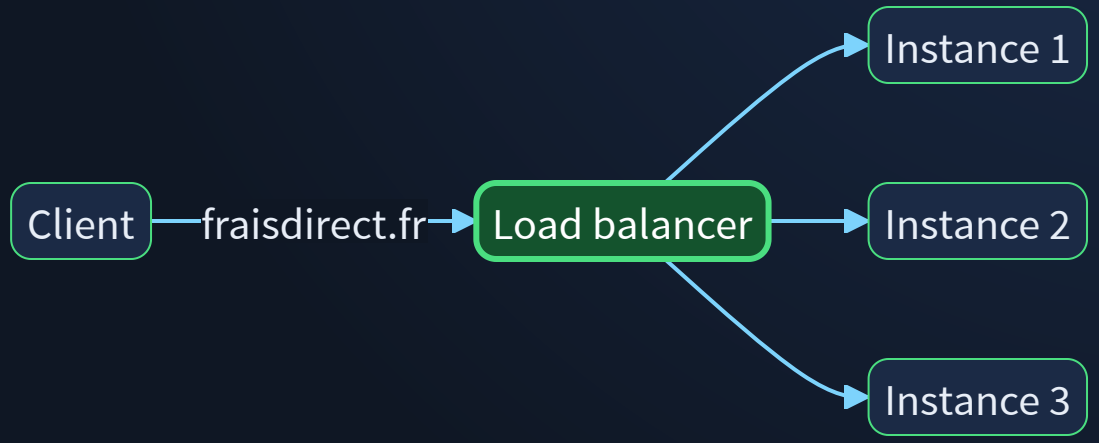
- 10 fois plus de clients que d'habitude
- Un seul serveur : il ne répond plus

Étape 1 : lancer plusieurs instances



Mais le client ne connaît qu'une adresse : `fraisdirect.fr`.
Sur quelle instance doit-il aller ?

Étape 2 : un load balancer devant



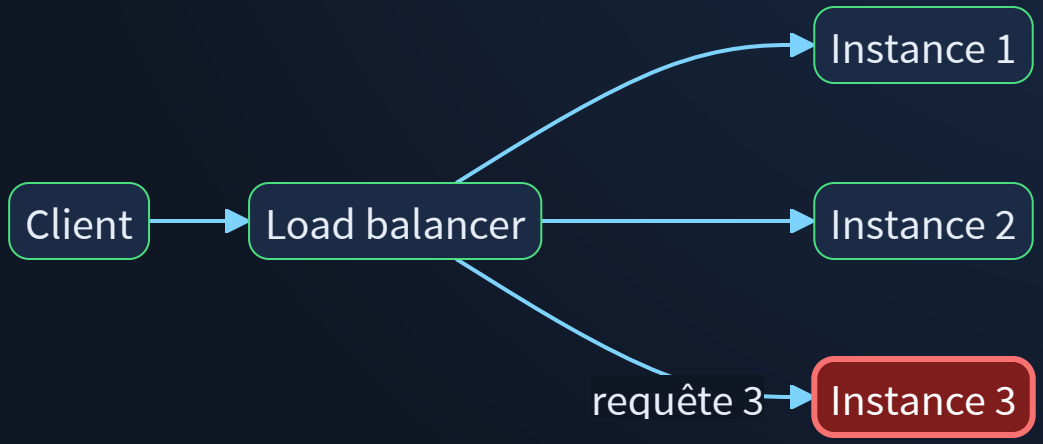
L'adresse `fraisdirect.fr` pointe sur le load balancer. Lui seul connaît les instances.

Étape 3 : répartir les requêtes

Le plus simple : chacun son tour (*round robin*)

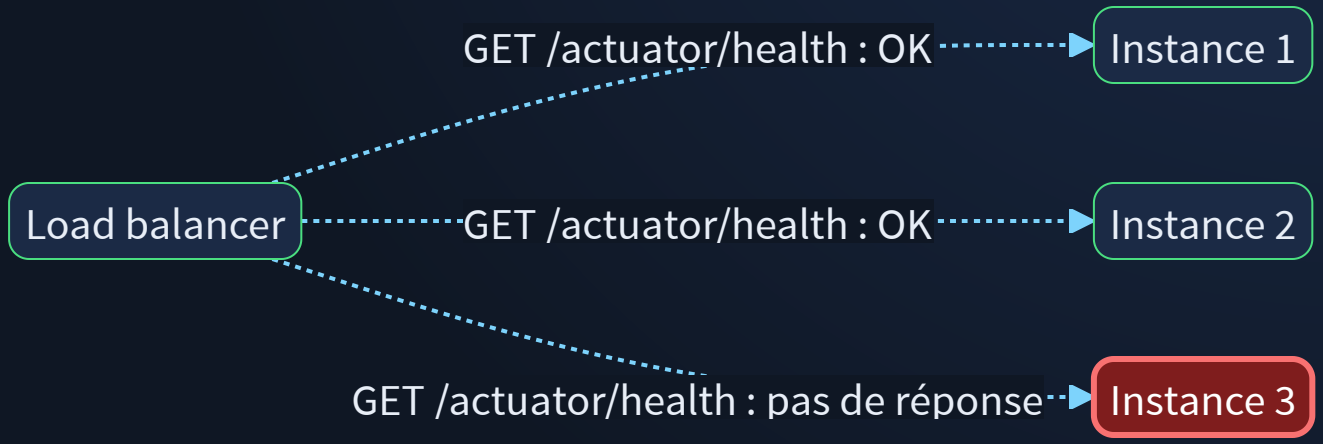
- Requête 1 → instance 1
- Requête 2 → instance 2
- Requête 3 → instance 3
- Requête 4 → instance 1 à nouveau

Et si une instance tombe ?



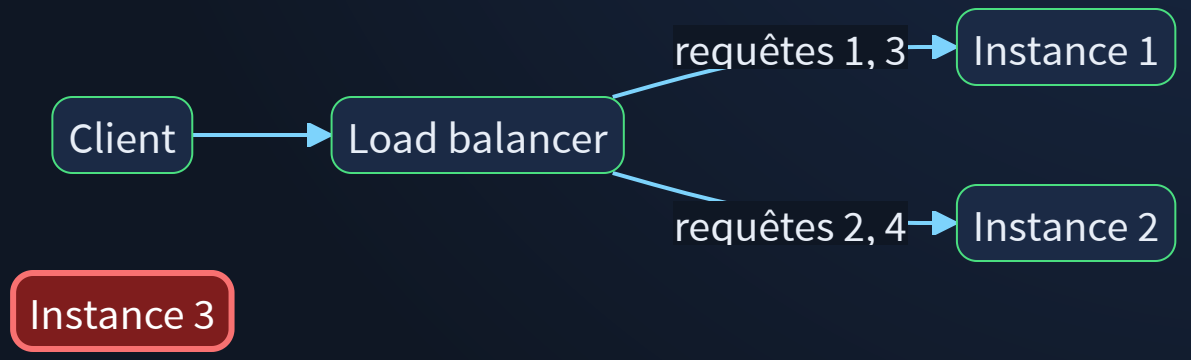
Une requête sur trois part vers une instance morte : le client a une erreur.

Étape 4 : le health check



Toutes les quelques secondes, le load balancer interroge chaque instance.

Étape 5 : retirer l'instance



Plus aucune requête ne part vers l'instance 3.
Quand elle répond de nouveau, le load balancer la réintègre.

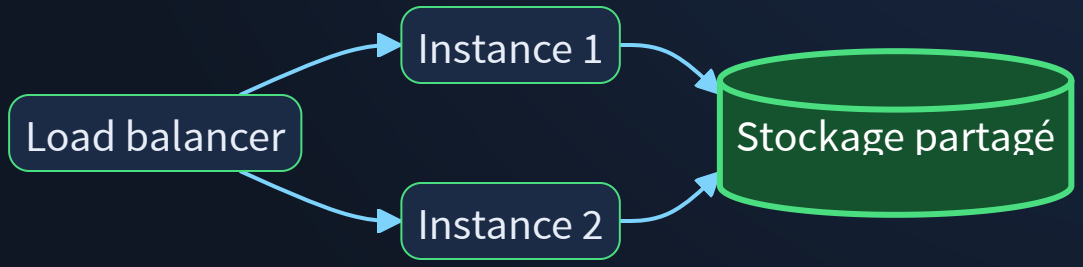
QUESTION

Le panier est gardé en mémoire dans la session HTTP de l'application. Avec 3 instances derrière un load balancer en round robin, ça marche ?

Le panier perdu, étape par étape

1. « Ajouter des tomates » → instance 1
   le panier est rangé dans la mémoire de l'instance 1
2. « Voir mon panier » → instance 2
3. L'instance 2 ne connaît pas ce panier : il est vide

Étape 6 : l'état hors de l'application



Les instances ne gardent rien en mémoire : elles sont **stateless**.

Quel stockage ? Réponse à l'acte 3.

À retenir

- Le load balancer est la seule adresse connue des clients
- Il répartit les requêtes entre les instances
- Il retire les instances qui ne répondent plus
- Les instances doivent être stateless

Exemples : HAProxy, Nginx, les load balancers des clouds

ACTE 2

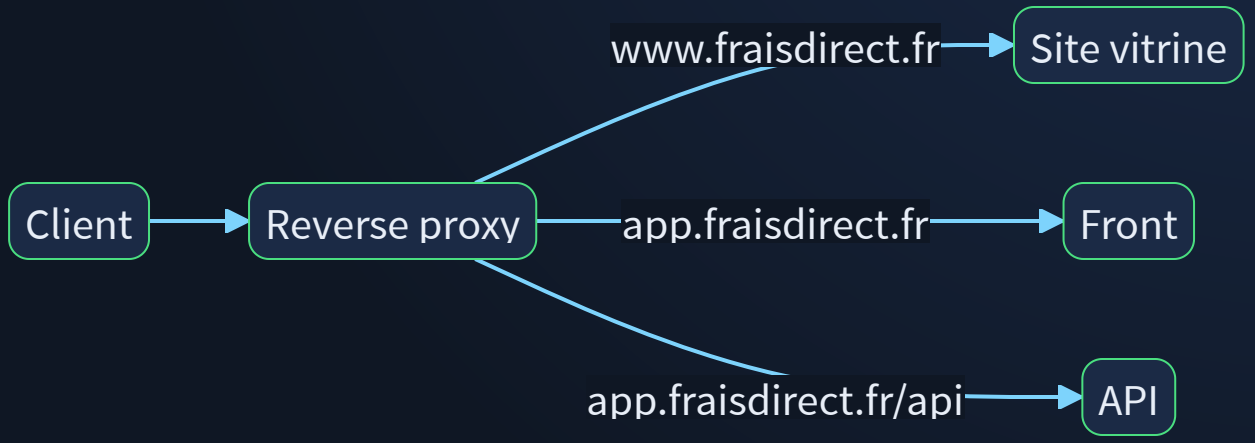
Un site, plusieurs applications

Une seule adresse pour tout le monde

Le problème

- FraisDirect a maintenant trois applications : un site vitrine, un front et une API
- Les clients ne doivent voir qu'une seule adresse

Le reverse proxy



Il reçoit toutes les requêtes et choisit l'application selon le domaine et le chemin.

Suivre une requête

`GET https://app.fraisdirect.fr/api/commandes`

1. Le reverse proxy reçoit la requête
2. Il lit le domaine : `app.fraisdirect.fr`
3. Il lit le chemin : commence par `/api`
4. Il transmet la requête à l'API
5. Il renvoie la réponse de l'API au client

La même chose avec Nginx

```
server {
    server_name app.fraisdirect.fr;

    location /api/ {
        proxy_pass http://api:8080;
    }

    location / {
        proxy_pass http://front:80;
    }
}
```

Bonus : HTTPS à un seul endroit

- Le client parle en HTTPS au reverse proxy
- Le reverse proxy déchiffre la requête
- Il la transmet en HTTP aux applications, sur le réseau interne

Un seul certificat à gérer, au lieu d'un par application.

Load balancer ou reverse proxy ?

- Souvent le même logiciel : Nginx fait les deux
- Le reverse proxy découpe le site en applications : préoccupation des DEV
- Le load balancer tient la charge et les pannes : préoccupation des OPS

Exemples : Nginx, Traefik, Caddy, HAProxy

QUESTION

Ajouter une 2e instance de l'application pour encaisser le pic du Black Friday : c'est d'abord le rôle…

A. du reverse proxy (préoccupation DEV)

B. du load balancer (préoccupation OPS)

ACTE 3

Des données de toutes les formes

La bonne base pour le bon besoin

Les besoins de FraisDirect

- Les commandes
- Le catalogue de produits
- Les paniers (promis à l'acte 1)
- La température des chambres froides
- Les photos des produits

La méthode

Pour chaque besoin, trois questions :

1. À quoi ressemblent les données ?
2. Comment les lit-on ?
3. Est-ce grave d'en perdre ?

Les commandes

commande	client	date
1042	Alice	24/11

commande	produit	quantité
1042	Tomates	2 kg
1042	Comté	300 g

1. Des lignes et des colonnes, liées entre elles
2. On les croise : les commandes d'un client, avec leurs lignes
3. Très grave : c'est de l'argent

Commandes : base relationnelle

- Des tables liées entre elles, interrogées en SQL
- ⊕ Transactions : tout est enregistré, ou rien
- ⊕ La base refuse les données incohérentes
- ⊖ Changer la structure demande une migration

PostgreSQL, MySQL / MariaDB, Oracle

Le catalogue

```
{ "nom": "Comté 18 mois", "prix": 32.5, "affinage": "18 mois",
{ "nom": "Tomate cœur de bœuf", "prix": 4.2, "origine": "Prove
```

1. Chaque produit a ses propres champs
2. On cherche par mots : « tomate provence », même avec une faute
3. Peu grave : on peut le reconstruire

Catalogue : base document

- Chaque donnée est un document JSON
- ⊕ Chaque document a ses propres champs
- ⊕ Recherche avancée (fautes de frappe, filtres)
- ⊖ Pas de jointures : on duplique les données

MongoDB, Elasticsearch

Les paniers

```
panier:alice → { "tomates": "2 kg", "comté": "300 g" }
panier:bob   → { "pain": 1 }
```

1. Une clé, une valeur
2. Toujours lu par sa clé, très souvent, très vite
3. Pas très grave : le client refait son panier

Paniers : base clé-valeur

- Comme une `Map`, partagée entre les instances
- ⊕ Très rapide : les données sont en mémoire
- ⊕ Une clé peut expirer toute seule (panier abandonné)
- ⊖ On ne lit que par la clé : pas de recherche

Redis / Valkey, Memcached

QUESTION

La température d'un capteur, relevée toutes les 10 secondes pendant un an : quel type de BDD ?

A. Relationnel

B. Document

C. Timeseries

D. Clé-valeur

Les chambres froides

heure	capteur	température
10:00:00	frigo-1	3,1 °C
10:00:10	frigo-1	3,2 °C
10:00:20	frigo-1	3,2 °C

1. Une mesure datée, encore et encore : 3 millions par an et par capteur
2. Par période : « la moyenne par heure cette semaine »
3. Une mesure perdue n'est pas grave

Chambres froides : base timeseries

- Faite pour des mesures rangées dans le temps
- ⊕ Compresse énormément les mesures
- ⊕ Calcule vite des moyennes par période
- ⊖ Ne sait faire que ça

InfluxDB, TimescaleDB, Prometheus

Les photos des produits

```
fraisdirect-photos/tomate-coeur-de-boeuf.jpg
fraisdirect-photos/comte-18-mois.jpg
```

1. Des fichiers, parfois gros
2. On les télécharge en entier, par leur nom
3. Gênant : il faut les sauvegarder

Photos : object storage

- Des fichiers (*objets*) rangés dans des *buckets*
- ⊕ Accessible en HTTP depuis toutes les instances
- ⊕ Pratiquement sans limite de taille
- ⊖ On remplace un fichier, on ne le modifie pas

AWS S3, Google Cloud Storage, Garage, Ceph

Récapitulatif

Besoin	Type	Exemple
Commandes	Relationnel	PostgreSQL
Catalogue	Document	MongoDB, Elasticsearch
Paniers	Clé-valeur	Redis
Températures	Timeseries	InfluxDB
Photos	Object storage	S3

QUESTION

Une application avec peu de trafic, beaucoup de règles métier et une recherche par nom de produit : quel type de SGBD choisir ?

A. Relationnel

B. Document

C. Clé-valeur

D. Timeseries

Une base de plus, c'est un coût

- Il faut l'installer, la sauvegarder, la surveiller, la mettre à jour
- PostgreSQL sait déjà chercher du texte et stocker du JSON
- On commence avec une base relationnelle, on en ajoute une autre quand le besoin est réel

Bonnes pratiques

- Une base ne se partage pas entre applications différentes
- Une base se sauvegarde… et on teste la restauration
- La bonne base pour le bon usage, mais pas une de plus que nécessaire

ACTE 4

Des services qui doivent se parler

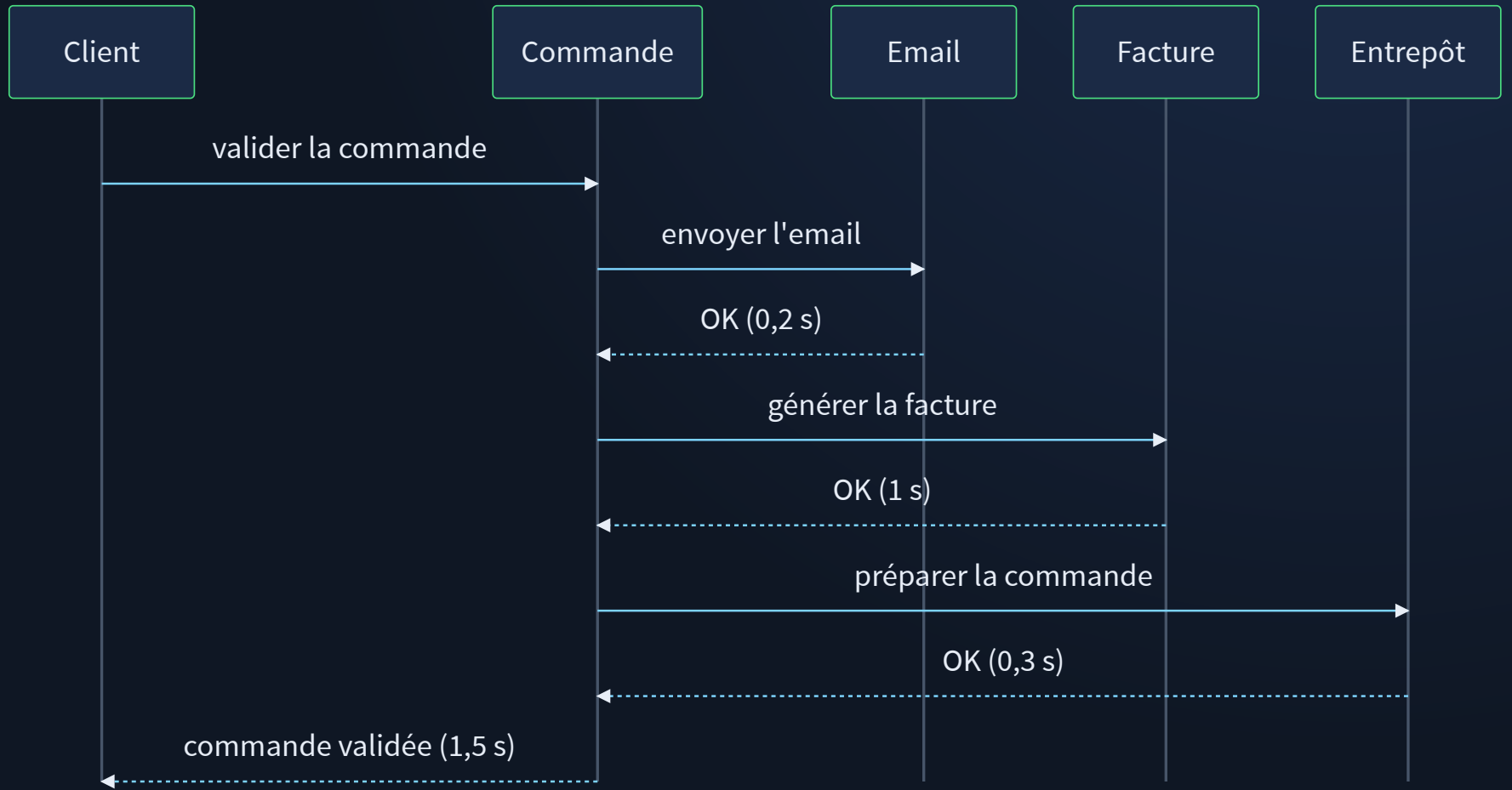
Le messaging

Le problème

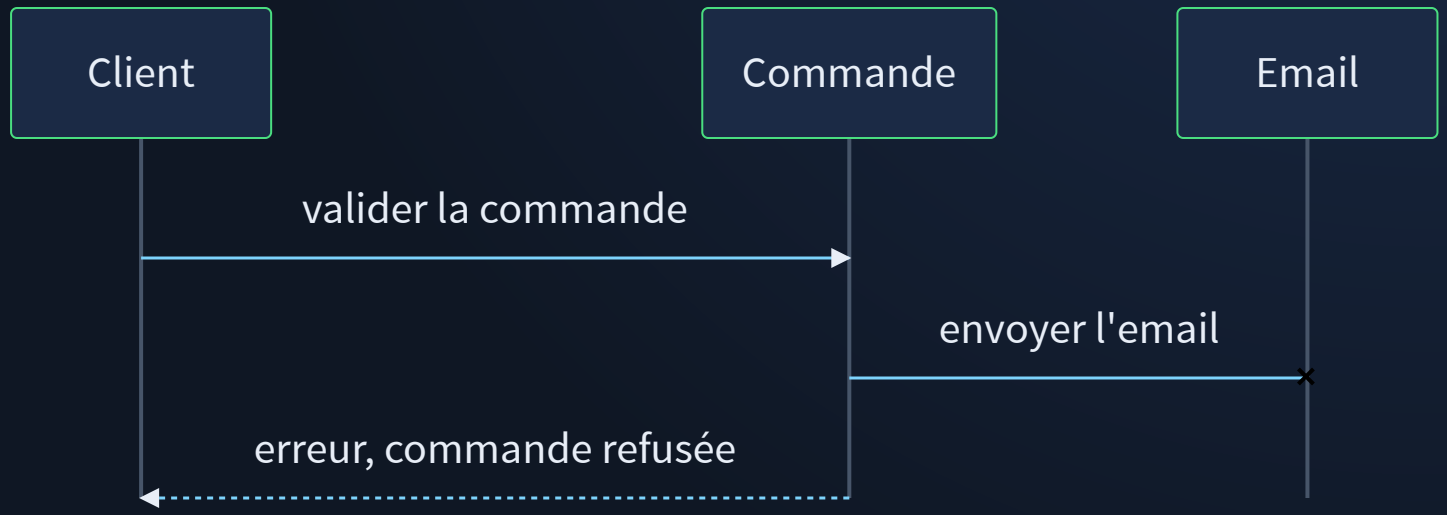
Quand une commande est validée, il faut aussi :

- envoyer un email de confirmation
- générer la facture
- prévenir l'entrepôt

Première idée : des appels HTTP

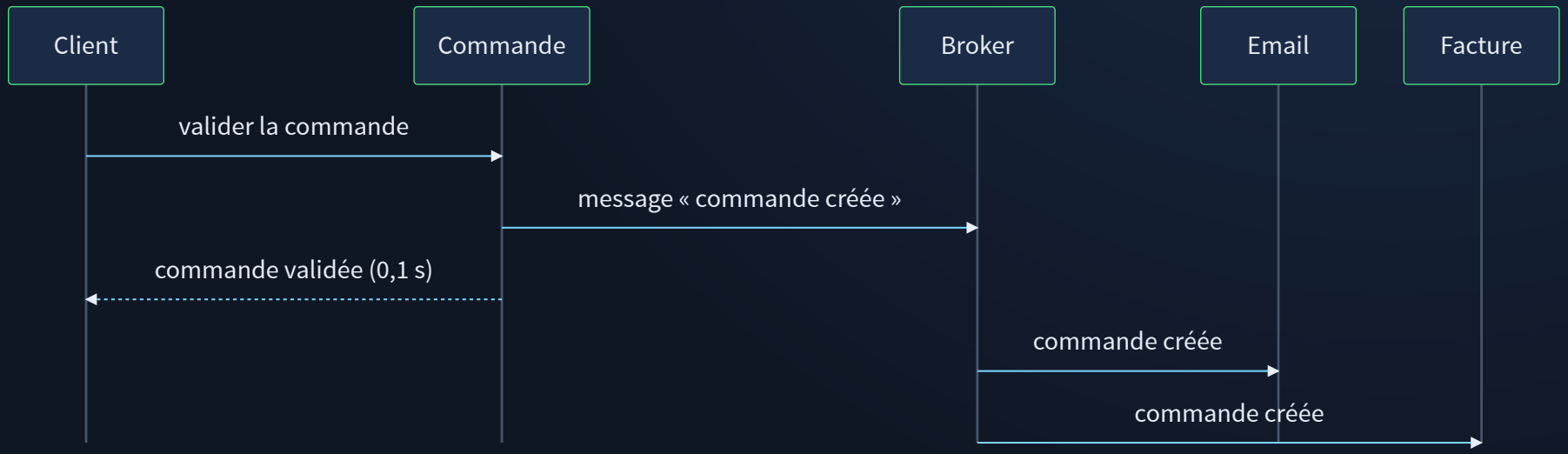


Et si l'email est en panne ?



Une panne de l'email empêche de vendre !

Deuxième idée : un broker de messages



Ce qu'on gagne

- **Rapide** : le client n'attend plus les autres services
- **Robuste** : si l'email est en panne, son message l'attend dans le broker
- **Évolutif** : un nouveau service s'abonne sans toucher au service commande

Bonus : absorber les pics



Pendant le pic, la file se remplit. Après, le service facture rattrape son retard. Rien n'est perdu.

Le vocabulaire



- **Producer** : envoie le message
- **Exchange** : le tri postal, il distribue les messages
- **Queue** : la boîte aux lettres, elle garde les messages
- **Consumer** : lit les messages d'une queue

Le trajet d'un message (1/5)



Le producer envoie le message à l'**exchange**, avec la routing key `commande.creee`.

Le trajet d'un message (2/5)



L'exchange compare la routing key aux **bindings** de chaque queue.

Le trajet d'un message (3/5)



Il dépose **une copie** du message dans chaque queue qui correspond.

Le trajet d'un message (4/5)



Chaque consumer lit le message dans sa queue et le traite.

Le trajet d'un message (5/5)



Le traitement a réussi : le consumer confirme (**ack**), la queue supprime le message.

La routing key

Des mots séparés par des points :

`commande.creee.fr`

Mot 1	Mot 2	Mot 3
commande	creee	fr
quoi	quel événement	quel pays

Le binding : mot à mot

Binding `commande.creee.fr`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
creee	creee	✓
fr	fr	✓

Tous les mots correspondent : la queue reçoit le message.

Le joker `*` : exactement un mot

Binding `commande.*.fr`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
*	creee	✓ un mot
fr	fr	✓

Cette queue reçoit tous les événements des commandes françaises.

Le joker `#` : zéro, un ou plusieurs mots

Binding `commande.#`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
#	creee.fr	✓ deux mots

Cette queue reçoit tout ce qui commence par `commande`.

QUESTION

Une queue est liée avec le binding `commande.*`. Quels messages reçoit-elle ?

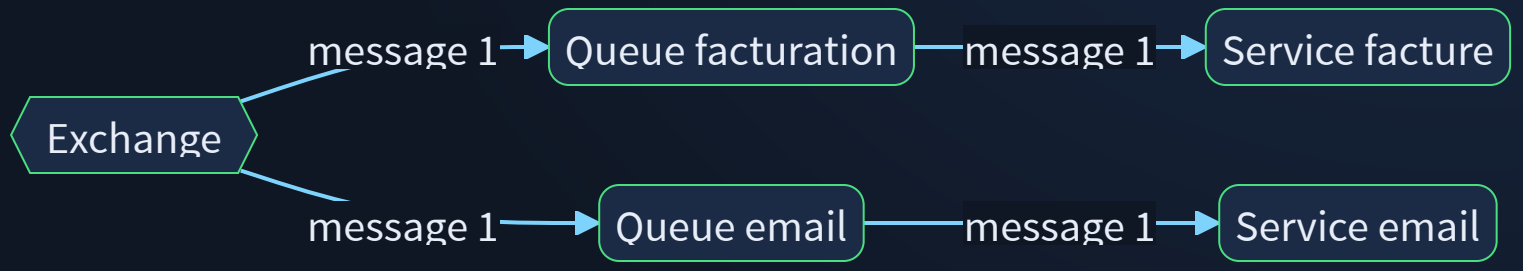
A. `commande.creee` seulement

B. `commande.creee.fr` seulement

C. Les deux

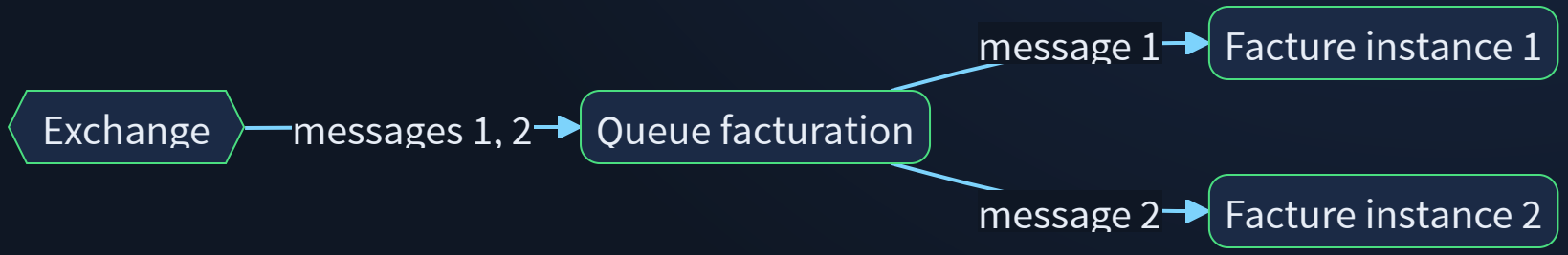
D. Aucun des deux

Plusieurs queues : chacun sa copie



Deux services différents : chacun reçoit le message.

Une queue, plusieurs instances : on se partage



Deux instances du même service : chaque message est traité **une seule fois**.

Des brokers

- RabbitMQ : celui de ce cours
- Mosquitto : pour les objets connectés (les capteurs des chambres froides)
- Kafka : pour de très gros volumes

ACTE 4 · PRATIQUE

RabbitMQ dans Spring

En 7 étapes

Ce qu'on va construire



1. La dépendance
2. Les messages en JSON
3. L'exchange
4. La queue
5. Le binding
6. Envoyer un message
7. Recevoir un message

Étape 1 : la dépendance

```
implementation 'org.springframework.boot:spring-boot-starter-a
```

Et l'adresse du broker :

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
```

Étape 2 : les messages en JSON

```
@Bean
MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}
```

Spring transforme nos objets Java en JSON à l'envoi, et le JSON en objets à la réception.

Étape 3 : l'exchange

```
@Bean
TopicExchange fraisDirectExchange() {
    return new TopicExchange("fraisdirect");
}
```

Un exchange de type *topic* : il comprend les jokers `*` et `#`.

Étape 4 : la queue

```java
public static final String FACTURATION_QUEUE = "facturation";

@Bean
Queue facturationQueue() {
    return QueueBuilder.durable(FACTURATION_QUEUE).build();
}
```

durable : la queue survit au redémarrage de RabbitMQ.

Étape 5 : le binding

```
@Bean
Binding facturationBinding(
        Queue facturationQueue,
        TopicExchange fraisDirectExchange) {
    return BindingBuilder.bind(facturationQueue)
            .to(fraisDirectExchange)
            .with("commande.creee.*");
}
```

La queue `facturation` reçoit toutes les commandes créées, quel que soit le pays.

Étape 6 : envoyer un message

```java
@Service
public class CommandeService {

    private final RabbitTemplate rabbitTemplate;
    // ...

    public void valider(Commande commande) {
        commandeRepository.save(commande);
        rabbitTemplate.convertAndSend("fraisdirect",
                "commande.creee." + commande.pays(),
                new CommandeCreee(commande.id()));
    }
}
```

Étape 7 : recevoir un message

```java
@Service
public class FacturationListener {

    private final FactureService factureService;
    // ...

    @RabbitListener(queues = FACTURATION_QUEUE)
    public void commandeCreee(CommandeCreee commande) {
        factureService.generer(commande);
    }
}
```

QUESTION

À vous : consommez la file `commandes`

Chaque message reçu (une chaîne) doit être passé à `commandeService.traiter(...)`.

ACTE 4 · ERREURS

Quand le traitement échoue

Ne pas perdre de message

QUESTION

Avec Spring AMQP et une queue classique, que devient par défaut un message dont le traitement lève une exception dans le listener ?

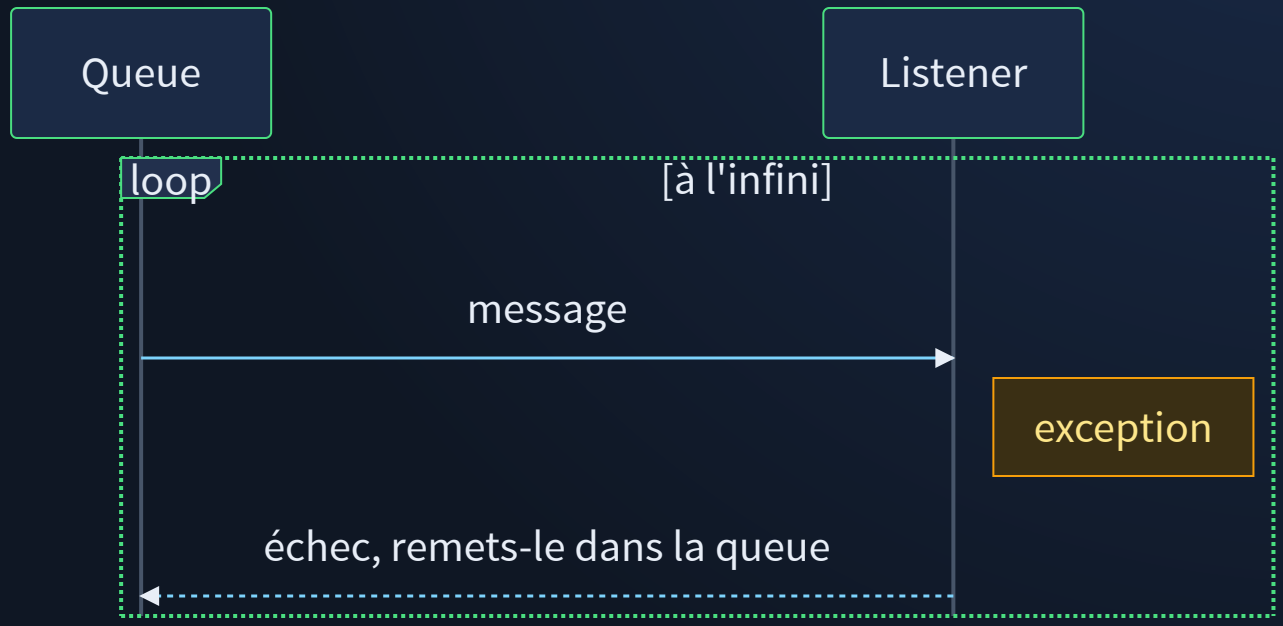
A. Il est perdu

B. Il est rejoué à l'infini

C. Il part dans une dead letter queue

D. Il est renvoyé au producer

Ce qui se passe par défaut



Si l'erreur vient d'un bug, le message échouera toujours : il tourne en boucle.

Solution, étape 1 : limiter les essais

```
spring.rabbitmq.listener.simple.retry.enabled=true
spring.rabbitmq.listener.simple.retry.max-attempts=3
```

- Spring essaie 3 fois
- Puis il rejette le message pour de bon
- … et le message est **supprimé** !

Solution, étape 2 : la dead letter queue



Un message rejeté est envoyé à un exchange dédié, qui le range dans une queue à part.

Spring : où envoyer les rejets

```
@Bean
Queue facturationQueue() {
    return QueueBuilder.durable(FACTURATION_QUEUE)
            .deadLetterExchange("fraisdirect.dlx")
            .build();
}
```

Spring : la queue des rejets

```
@Bean
FanoutExchange deadLetterExchange() {
    return new FanoutExchange("fraisdirect.dlx");
}

@Bean
Queue facturationDlq() {
    return QueueBuilder.durable("facturation.dlq").build();
}
```

Spring : relier les deux

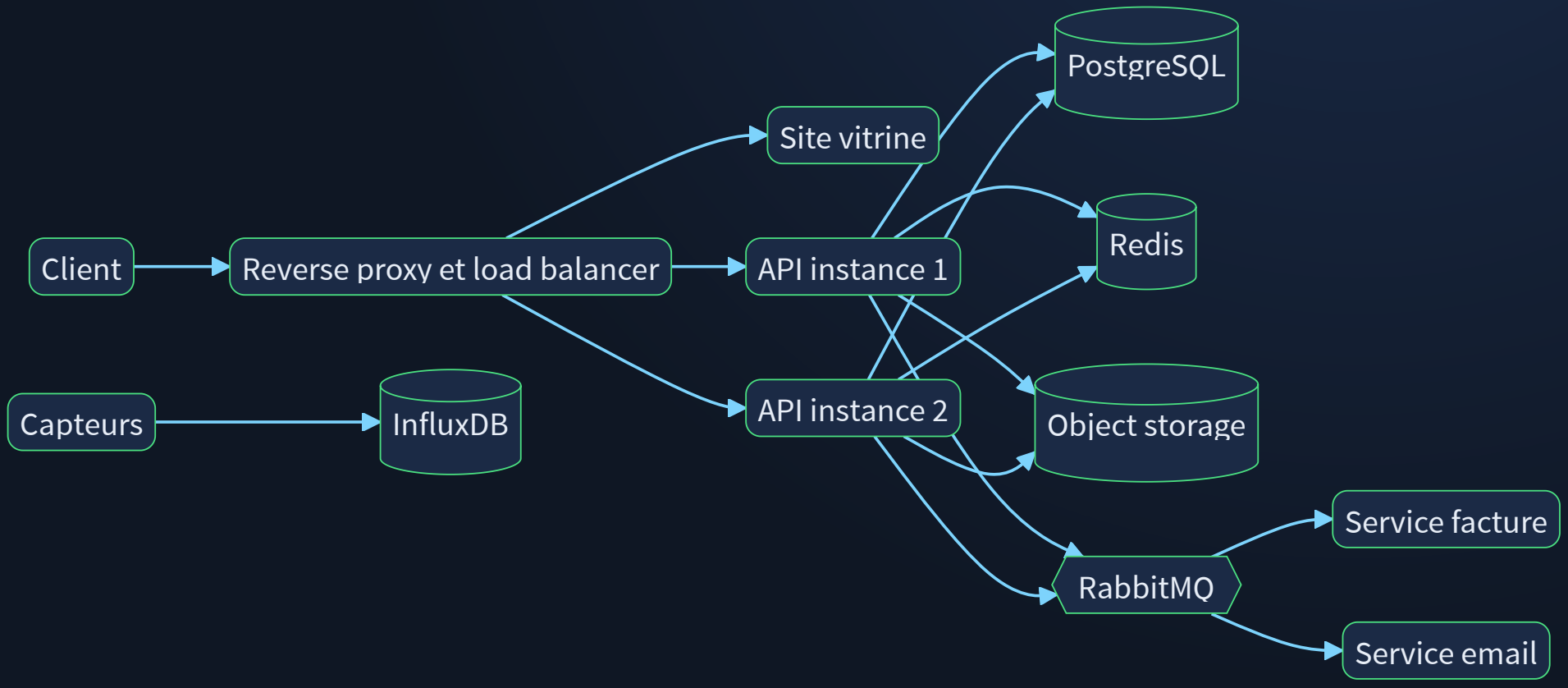
```
@Bean
Binding facturationDlqBinding(
        Queue facturationDlq,
        FanoutExchange deadLetterExchange) {
    return BindingBuilder.bind(facturationDlq)
            .to(deadLetterExchange);
}
```

Pas de `.with(...)` : un exchange fanout ne regarde pas la routing key.

Le parcours complet d'un message en échec

1. Le listener lève une exception
2. Spring réessaie, jusqu'à 3 fois
3. Spring rejette le message
4. RabbitMQ l'envoie à l'exchange `fraisdirect.dlx`
5. Le message attend dans `facturation.dlq`

FraisDirect aujourd'hui



À retenir

- Chaque middleware répond à un problème précis
- On l'ajoute quand le problème se pose, pas avant
- Chaque brique en plus doit être installée, sauvegardée et surveillée